



14.2. Selection Sort & Insertion Sort

[Previous Section:](#)

 [14.1. Bubble Sort & Comb Sort](#)

Contents:

[1. Selection Sort](#)

[2. Insertion Sort](#)

[Summary](#)

[References](#)

In the previous section, we studied **Bubble Sort** and **Comb Sort**, two algorithms that repeatedly compare and swap elements until the array becomes sorted.

Although they work well for understanding the basic concepts of sorting, both algorithms may perform many unnecessary swaps before every element reaches its correct position.

In this section, we'll explore two new approaches.

Instead of repeatedly swapping neighboring elements, these algorithms answer two different questions:

- **Selection Sort:** *"Which element should be placed here next?"*
- **Insertion Sort:** *"Where should this new element be inserted?"*

Although both algorithms still have a worst-case time complexity of $O(n^2)$, they introduce two important ideas that appear repeatedly in more advanced sorting algorithms: **selection** and **insertion**.

1. Selection Sort

Bubble Sort repeatedly compares **adjacent elements**. Selection Sort takes a different approach. Instead of looking at neighboring elements, it focuses on **one position at a time**.

The idea is very simple. Suppose we are trying to place the smallest element at the beginning of the array. Instead of swapping every time we find a smaller value, we scan the remaining portion of the array to determine which element is the smallest.

Once the smallest element has been found, we place it into its correct position. Then we repeat the process for the second position, the third position, and so on.

In other words, during each iteration, Selection Sort selects the smallest element from the unsorted portion and places it into its final position.

Consider the following array.

29	72	98	13	87	66	52	51	36
----	----	----	----	----	----	----	----	----

- Iteration 1: Assume that **29** is currently the smallest value. We compare it with every remaining element.
 - 29 ? 72
 - 29 ? 98
 - 29 ? 13 ← smaller
 - 29 ? 87
 - 29 ? 66
 - 29 ? 52
 - 29 ? 51
 - 29 ? 36

The smallest value is **13**. Swap **29** and **13**.

29	72	98	13	87	66	52	51	36
----	----	----	----	----	----	----	----	----

13	72	98	29	87	66	52	51	36
----	----	----	----	----	----	----	----	----

↑
Sorted

The first position is now correct.

- Iteration 2: Now ignore the first element. Assume **72** is the smallest in the remaining portion. Compare it with every remaining element.
 - 72 ? 98
 - 72 ? 29 ← smaller
 - 29 ? 87
 - 29 ? 66
 - 29 ? 52
 - 29 ? 51
 - 29 ? 36

Swap **72** and **29**.

13	72	98	29	87	66	52	51	36
----	----	----	----	----	----	----	----	----

13	29	98	72	87	66	52	51	36
----	----	----	----	----	----	----	----	----

↑ ↑
Sorted

The algorithm continues until every position has been processed.

13	29	36	72	87	66	52	51	98
----	----	----	----	----	----	----	----	----

13	29	36	51	87	66	52	72	98
----	----	----	----	----	----	----	----	----

13	29	36	51	52	66	87	72	98
----	----	----	----	----	----	----	----	----

No swap is performed at this step since it remains the minimum value:

13	29	36	51	52	66	87	72	98
----	----	----	----	----	----	----	----	----

13	29	36	51	52	66	72	87	98
----	----	----	----	----	----	----	----	----

13	29	36	51	52	66	72	87	98
----	----	----	----	----	----	----	----	----

- Number of Comparisons: Notice that every iteration compares one element with every remaining unsorted element.

Iteration	Comparisons
0	$n - 1$
1	$n - 2$
2	$n - 3$
...	...
$n - 2$	1

Therefore, $[(n - 1) + (n - 2) + \dots + 1 = \frac{n(n-1)}{2}]$, which gives Time Complexity = $O(n^2)$

Unlike Bubble Sort, however, Selection Sort performs **far fewer swaps**. In fact, each iteration performs **at most one swap**, making it useful when swapping elements is expensive.

- Complexity

Complexity	Value
Best Case	$O(n^2)$
Average Case	$O(n^2)$

Complexity	Value
Worst Case	$O(n^2)$
Space Complexity	$O(1)$

- Implementation

```
def selection_sort(a): # Selection Sort [Time(n), Space(n)
                        = O(n ^ 2), O(1)]
    n = len(a)
    for i in range(0, n - 1, 1):
        # Suppose the element at index i is the min value,
        # flag it
        flag = i
        for j in range(i + 1, n, 1):
            # Find the minimum element and flag it
            if a[j] < a[flag]: # current element < the fla
                                g element
                flag = j
        # Rotate the current element with the ith element
        a[i], a[flag] = a[flag], a[i]
    return a
```

2. Insertion Sort

So far, the sorting algorithms we've studied focus on **swapping** elements. Insertion Sort takes a completely different approach.

Instead of repeatedly swapping values, it **inserts** each new element into its correct position within an already sorted portion of the array.

A good way to visualize this is how we organize a hand of playing cards.

- Suppose we already have several cards arranged in ascending order. When we receive a new card, we don't sort the entire hand again.
- Instead, we compare the new card with the existing cards, shift some cards to make room if necessary, and insert the new card into its correct position.

Insertion Sort works exactly the same way. Initially, the sorted portion contains only **one element**, since a single element is already sorted by itself. Then, one

by one, each remaining element is inserted into the correct position, causing the sorted portion to grow after every iteration.

Consider the following array.

85	12	59	45	72	51
----	----	----	----	----	----

- Initially, only the first element is considered sorted.

Sorted | Unsorted

85	12	59	45	72	51
----	----	----	----	----	----

- Iteration 1: Take the next element (**12**) as the **key**. Compare it with the sorted portion from **right to left**.

12 < 85 -> Shift 85 to the right

	85	59	45	72	51
--	----	----	----	----	----

There are no more elements to compare. Insert 12 into the empty position.

12	85	59	45	72	51
----	----	----	----	----	----

The first two elements are now sorted.

- Iteration 2: Take the next element (**59**). Compare from right to left.

59 < 85 -> Shift 85.

12		85	45	72	51
----	--	----	----	----	----

59 > 12 -> Stop shifting and insert 59.

12	59	85	45	72	51
----	----	----	----	----	----

- Iteration 3: Take **45**. Compare from right to left.

45 < 85 -> Shift 85.

12	59		85	72	51
----	----	--	----	----	----

45 < 59 -> Shift 59.

12		59	85	72	51
----	--	----	----	----	----

45 > 12 -> Stop shifting. Insert 45.

12	45	59	85	72	51
----	----	----	----	----	----

- Iteration 4: Take **72**.

72 < 85 -> Shift 85.

12	45	59		85	51
----	----	----	--	----	----

72 > 59 -> Stop shifting. Insert 72.

12	45	59	72	85	51
----	----	----	----	----	----

- Iteration 5: Take **51**.

51 < 85 -> Shift 85

12	45	59	72		85
----	----	----	----	--	----

51 < 72 -> Shift 72

12	45	59		72	85
----	----	----	--	----	----

51 < 59 -> Shift 59

12	45		59	72	85
----	----	--	----	----	----

51 > 45 -> Stop shifting. Insert 51.

12	45	51	59	72	85
----	----	----	----	----	----

The array is now completely sorted.

- **Observations**

Notice that Insertion Sort **does not repeatedly swap elements**. Instead, it performs two simple operations:

1. Compare the **key** with the sorted portion from right to left.
2. Shift every larger element one position to the right until the correct position is found.

Once enough space has been created, the key is inserted into that position. As a result, the sorted portion grows by one element after every iteration.

- Complexity

Complexity	Value
Best Case	$O(n)$

Complexity	Value
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space Complexity	$O(1)$

Insertion Sort performs exceptionally well when the array is already sorted or nearly sorted. In the best case, each new element is already in its correct position, so only one comparison is required per iteration, giving a best-case time complexity of $O(n)$.

- **Implementation**

```
def insertion_sort(a): # Insertion Sort [Time(n), Space(n)
                        = O(n ^ 2), O(1)]
    n = len(a)
    # left: The left array with the elements sorted
    # right: The right array uses for shifting elements
    # Temp: Store the element of the right array
    for right in range(1, n, 1): # Start from index 1
        temp = a[right]
        left = right - 1
        while left >= 0 and a[left] > temp:
            # Shift the element to the right to perform replacement
            a[left + 1] = a[left]
            left -= 1
        a[left + 1] = temp # Right position found
    return a
```

Summary

Selection Sort and Insertion Sort are both **comparison-based sorting algorithms**, but they organize the array in very different ways.

- **Selection Sort** repeatedly scans the unsorted portion of the array to find the smallest element and places it into its final position. Although it performs the same $O(n^2)$ number of comparisons as Bubble Sort, it makes

significantly fewer swaps because each iteration requires at most one swap.

- **Insertion Sort**, on the other hand, gradually builds a sorted portion of the array by inserting each new element into its correct position. Instead of swapping repeatedly, it shifts larger elements to create space before inserting the current element. This approach makes Insertion Sort particularly efficient for **small or nearly sorted datasets**, where it can achieve a best-case time complexity of $O(n)$.

Together with Bubble Sort and Comb Sort, these four algorithms introduce the fundamental techniques of **swapping**, **selection**, and **insertion** that form the basis of many more advanced sorting algorithms. In the next section, we'll explore faster sorting techniques designed to efficiently handle much larger datasets.

References

https://drive.google.com/file/d/1jK22rfWszKKgPgeprquGoeJKqqBg8rUY/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/selection-sort-algorithm-2/>

<https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/>

<https://www.programiz.com/dsa/selection-sort>

<https://www.programiz.com/dsa/insertion-sort>

Next Section:

 [14.3. Radix Sort](#)